

sLLM 및 Fine Tuning 2일차 실습 서브 노트

Qwen2.5-1.5B-Instruct · LoRA SFT 실습

P231 임유리

학습한 내용 및 실습한 사항

README 설정으로 학습한 뒤 입력 길이, LoRA r/α , 학습률과 epoch를 바꿔 보았다. 학습과 평가가 끝난 아홉개의 실행 결과를 비교하였고, 숫자만 보고 판단하기는 어려워 같은 질문의 실제 답변도 함께 읽었다.

1. 이해관계자 가치(Pain Point)

AURORA 재택근무 질문의 답변에는 잘못된 등급과 없는 출근 조건이 섞여 있었다. 직원이 이런 답변을 믿고 신청하면 다시 규정을 찾아보거나 인사팀에 문의해야 한다. 인사 담당자도 잘못 안내된 내용을 정정해야 한다. 그래서 이 실습의 문제를 “그렇듯하게 답하는가”보다 “직원이 규정에 맞게 판단할 수 있는가”로 잡았다. 실제 회사에서 문의가 줄었는지 측정한 것은 아니며, 실습 답변에서 확인한 오류를 업무 상황에 연결한 것이다.

2. 이를 해결하기 위한 AI 솔루션

Human AI Corporation의 인사 규정 안내 도우미를 생각했다. 질문과 모범 답변으로 LoRA Adapter를 학습하고, 검색한 규정 원문을 답변에 함께 넣는 방식이다. SFT에는 답변 순서와 확인 안내를, RAG에는 질문에 필요한 근거를 말길 수 있다. 다만 이번 결과에는 오류가 남아 있어 정확한 규정 안내가 이미 완성됐다고 보기는 어렵다. 작은 모델을 로컬에서 실행하면 외부 서비스에 사내 질문을 보내지 않는 구성을 시도할 수 있다. 다만 실제 보안 요건 충족 여부는 이번에 검증하지 않았다.

- 입력 : 임직원의 HR 관련 자연어 질문
- 목표 기능 : 회사 규정과 절차 안내, 출처 확인, 규정에 없는 질문에는 담당 부서 확인 안내
- SFT Pipeline 역할 : 시스템 지시문 + 사용자 질문 + 기대 답변의 관계를 학습
- 기대하는 효과: 직원이 규정을 찾는 부담과 인사팀의 반복 설명을 줄이는 것. 실제 업무 효과는 아직 측정하지 않았다.

3. 아키텍처 구성도

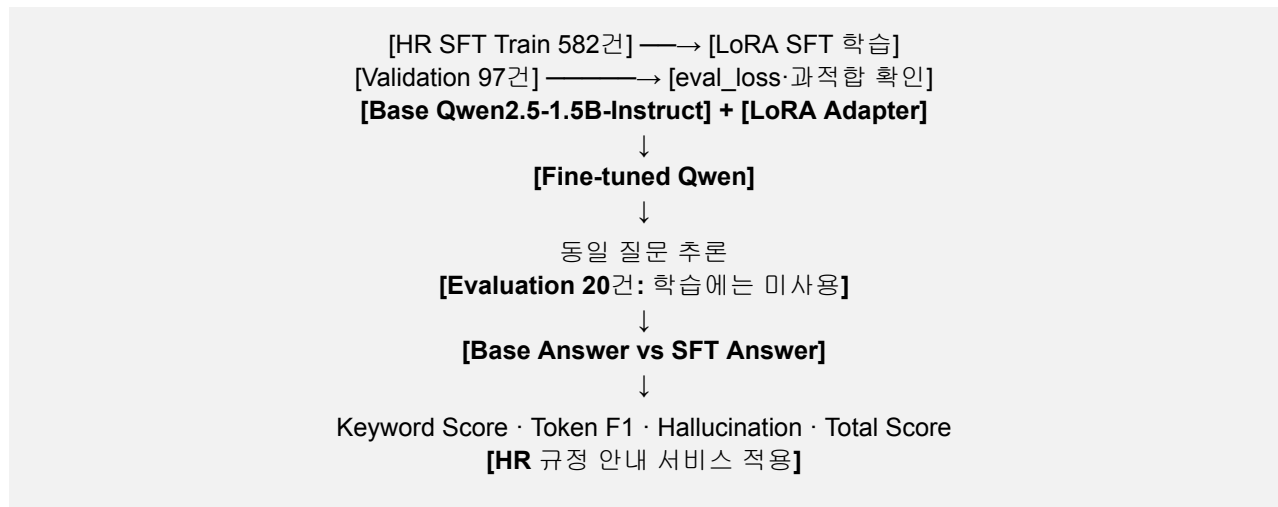


그림 1. 2일차 LoRA SFT 학습 및 평가 파이프라인

Train은 Adapter를 학습하는 데이터, Validation은 학습 중 loss를 확인하는 데이터, Evaluation은 학습 후 Base와 SFT 답변을 비교하는 데이터다. 이번에는 같은 Evaluation 20문항을 여러 설정에서 반복해서 확인했다. 학습 입력에 넣지는 않았지만 설정 선택에 참고했으므로, 새 질문에서도 같은 성능이 나는지는 별도로 확인해야 한다.

SFT와 RAG의 연결

규정 문서 → 문서 단위 정리 또는 **chunk** 분리 → 임베딩 → 검색 인덱스
 직원 질문 → 질문 임베딩 → 관련 규정 검색 → 질문 + 검색한 근거
 ↓
Base Qwen + 학습한 LoRA Adapter → 규정 답변 → 원문·예외 조건 확인

위 파이프라인은 SFT와 RAG를 함께 쓸 때의 연결이다. 로컬 FAISS 노트북은 문서 6개를 BGE-M3로 벡터화하고 Base Qwen만 사용한다.

README 설정으로 재학습

처음에는 max_length=512, epochs=3으로 실행했다. 그 후 README에 맞춰 384와 2로 바꾼 뒤 다시 학습했다. 시간은 줄었지만 두 값을 한꺼번에 바꿔서 길이와 반복 횟수 중 무엇이 더 영향을 줬는지는 알 수 없었다.

항목	이전 실행 기록	이번 README 재실행
max_length / epochs	512 / 3	384 / 2
총 학습 step	219	146
학습 시간	약 1시간 58분	3933초, 약 65분 33초
최종 train loss	0.426826	0.673540
마지막 eval loss	0.01887	0.217
20문항 평가	Base 1승 / FT 19승	Base 1승 / FT 18승 / 무승부 1건

실행 명령

README 설정으로 다시 실행한 명령이다. 이후에는 입력 길이, 학습률, `r/alpha`, `epoch`를 바꿔 가며 비교했다.

```
SFT_MAX_LENGTH=384 \  
SFT_EPOCHS=2 \  
SFT_LEARNING_RATE=1e-4 \  
python scripts/train_lora.py
```

Max_length : 한 샘플에서 사용하는 최대 토큰 길이. 줄이면 메모리·계산량을 줄일 수 있으나 긴 질문이나 답변이 잘릴 수 있다.

`epochs`는 전체 학습 데이터를 반복하는 횟수다. 실험에서는 2 epoch가 146 step, 3 epoch가 219 step이었다. `learning_rate`는 학습 중 가중치를 고치는 크기와 관련된다. README 재실행은 `1e-4`였고, 이후 `5e-5`로 낮춘 결과도 비교했다. 또한 `gradient accumulation`은 8로 유지했다. LoRA는 `r8/alpha16`, `r16/alpha32`, `r32/alpha64`를 비교했다. `r`을 바꿀 때 `alpha`도 함께 조정했으므로 두 값을 한 묶음으로 기록했다.

384 설정의 학습 결과

2 epoch 학습 완료 후 최종 `train loss 0.673540`, `eval loss 0.217`, `eval mean token accuracy 0.9571`를 확인했다. 시간이 줄었지만 길이와 반복 횟수를 동시에 바꿨으므로 어느 하나의 효과라고 단정할 수 없다고 볼 수 있다. 낮은 `loss`만으로 실제 답변의 근거성과 품질까지 보장되지 않는다는 점도 평가 답변에서 확인했다.

max_length 256 비교 실험

384 실행과 256 실행, 두 `training_summary.json`을 대조하니 `max_length`만 384→256으로 달랐고, 기록된 `epochs=2`, 학습률 `1e-4`, 누적 8, `r=16`, `alpha=32`, MPS 모드는 같았다. 다만 앱 종료와 절전 모드 해제도 함께 했으므로 환경까지 통제된 실험은 아니다. 이번 실험에서 학습에 1시간 이상 걸려 다음 실험에서 `max_length`를 384에서 256으로 줄여 보았다. 시간이 얼마나 줄어드는지, 평가 점수와 답변이 어떻게 달라지는지 함께 비교했다.

항목	384 실행	256 실행
최종 <code>train loss</code>	0.673540	0.551308
마지막 <code>eval loss</code>	0.217014	0.089834
<code>eval token accuracy</code>	0.9571	0.9890
학습 시간	65분 33초	25분 50초
완료 <code>step</code>	146/146	146/146

256 실행은 약 39분 43초 짧아졌고, eval loss는 1 epoch의 0.307224에서 2 epoch의 0.089834로 내려갔다. 길이에 따라 포함되는 토큰도 달라져 loss만으로 긴 답변을 더 잘 배웠다고 볼 수는 없다. 캡처의 0.2647은 eval entropy다.

256 실행 학습 완료: 2 epoch, 146 step, 25분 50초. 정확한 eval loss는 training_log.json에서 확인했다.

384 실행 기록

실행 항목	실제 확인값
실행 환경	macOS arm64 / PyTorch 2.13.0 / MPS
학습 모드	FP32 LoRA Apple MPS
학습 설정	Max length 384 / Epochs 2 / Learning rate 0.0001
LoRA 설정	rank 16 / alpha 32 / dropout 0.05 / gradient accumulation 8
데이터	Train 582건 / Validation 97건
최종 손실	train_loss 0.673540 / eval_loss 0.217 eval_mean_token_accuracy 0.9571
학습 시간	약 3,933초(65분 33초) / 총 146/146 step
저장 결과	models/hr-qwen-lora / checkpoint-146 0.296 samples/sec · 0.037 steps/sec

앞선 256·r16 결과와 비교

항목	256·2 epoch·r16	512·3 epoch·r8
학습 시간	25분 50초	약 52분 58초
Adapter 가중치 파일	73,911,112 bytes	36,981,072 bytes
Keyword	0.3050	0.2350
Token F1	0.2435	0.2779
Total	0.3303	0.3136
Base 상대 승패	19승·0패·1무	19승·0패·1무
Unknown 자동 환각	0/5	0/5

Adapter 파일 크기는 약 절반으로 줄었다. 이것은 두 ZIP의 adapter_model.safetensors 크기를 직접 비교한 결과이며 전체 Base 모델이나 실행 메모리가 절반이라는 뜻은 아니다. 평균 F1은 0.0344 높아졌지만 Keyword는 0.0700, Total은 0.0167 낮아졌다. 학습 시간 증가 역시 r 때문이라고 단정할 수 없다고 볼 수 있다.

```
Token F1      : 0.0964
Total score   : 0.0862
Hallucination : 0/5

[Fine-tuned Model]
Keyword score : 0.235
Token F1      : 0.2779
Total score   : 0.3136
Hallucination : 0/5

[Improvement]
Keyword       : +0.1700
Token F1      : +0.1815
Total         : +0.2275
Hallucination : +0.0000

Detailed results : /Users/yurilim/Downloads/sllm-main-std/outputs/evaluation_results.jsonl
Summary          : /Users/yurilim/Downloads/sllm-main-std/outputs/evaluation_summary.json

[Evaluation] Complete
[Evaluation] Runtime resources released
```

r8 실행 평가 완료 캡처로, 승패는 evaluation_summary.json에서 19승·0패·1무로 확인했다.

답변을 읽고 확인한 점

AURORA 답변은 “AURARA-A부터 AURORAA-N까지 10단계”라고 잘못 설명했고 일본어 혼합과 질문에 무관한 영수증 이야기가 남았다. 이 문항 Total은 0.0486→0.0622로 올랐지만 핵심 등급 키워드는 여전히 모두 빠졌다. 개인 명품 구매비 질문에서도 규정에 없다고 한 뒤 지원하지 않는다고 단정하고 여러 언어를 섞었다. 해당 문항 Total은 0.7370→0.6927로 내려갔다. 자동 환각 수치는 0/5여도 실제 답변은 별도로 검토해야 했다.

설정을 다시 맞춰 재실행

r과 alpha를 바꿔 실행했지만 저장된 설정은 길이 512·3 epoch였다. 다음에는 max_length=256, epochs=2, r=8, alpha=16, 학습률 1e-4를 지정해 다시 돌렸다(outputs 4.zip). 코드를 수정한 뒤에도 실제 적용된 값은 시작 로그에서 다시 확인해야 했다.

길이·학습률·epoch·r/alpha를 바꾼 기록

길이 256과 2 epoch를 맞춘 뒤 r8/alpha16에서 학습률 1e-4와 5e-5를 비교했다. 그다음에는 5e-5를 유지하고 r16/alpha32로 되돌렸다. 다음 설정을 고를 때 도움을 받았고 학습과 평가는 직접 실행했다. 아래에는 실행 후 결과를 비교한 내용을 적었다.

자료	길이 / epoch	학습률	r / alpha	시간 / step
outputs.zip	384 / 2	1e-04	16 / 32	65분 33초 / 146
outputs 2.zip	256 / 2	1e-04	16 / 32	25분 50초 / 146
outputs 3.zip	512 / 3	1e-04	8 / 16	52분 58초 / 219
outputs 4.zip	256 / 2	1e-04	8 / 16	24분 26초 / 146
outputs 5.zip	256 / 2	5e-05	8 / 16	25분 16초 / 146
outputs 6.zip	256 / 2	5e-05	16 / 32	24분 23초 / 146
outputs 7.zip	256 / 3	5e-05	16 / 32	34분 52초 / 219
outputs 8.zip	256 / 3	1e-04	16 / 32	33분 13초 / 219
outputs 9.zip	256 / 3	1e-04	32 / 64	34분 19초 / 219

공통 기록: Qwen2.5-1.5B-Instruct, MPS·FP32 LoRA, 누적 8, 학습 582건·검증 97건. 시간은 training_log.json의 train_runtime을 초 단위 반올림했다. 최초 512·3 epoch·train loss 0.426826 기록은 이 아홉 ZIP보다 이전 실행이다.

자료 번호	최종 train loss	마지막 eval loss	eval token accuracy
1	0.673540	0.217014	0.9571
2	0.551308	0.089834	0.9890
3	0.580301	0.103726	0.9849
4	0.749842	0.320161	0.9324
5	1.039371	0.720923	0.8360
6	0.853360	0.482824	0.8906
7	0.583424	0.139433	0.9763
8	0.354490	0.007838	1.0000
9	0.277076	0.003532	1.0000

r16/alpha32 → r8/alpha16 : outputs 2와 4를 비교하면 시간은 25분 50초에서 24분 26초로 약 1분 24초 줄었다. 반면 eval loss는 0.089834에서 0.320161로 높아졌다. 이 한 번의 실행에서는 작은 Adapter가 시간을 조금 줄였지만 점수까지 유지하지는 못했다. 학습률 1e-4 → 5e-5: r8/alpha16을 유지한 outputs 4와 5에서 train loss는 0.749842→1.039371, eval loss는 0.320161→0.720923으로 높아졌다. 작은 학습률이면 무조건 나아질 것이라고 볼 수 없었다. 2 epoch 안에 충분히 배우지 못했을 가능성은 있지만, 아직 원인으로 확정하지 않았다.

낮은 학습률에서 **r16/alpha32**로 복귀 : outputs 5와 6에서는 eval loss가 0.720923→0.482824로 낮아졌다. 다만 원래 r16/alpha32·1e-4 실행의 0.089834보다는 높았다. r/alpha와 학습률 중 어느 조합이었는지 같이 봐야 했다.

2 epoch에서 3 epoch로 늘린 결과

5e-5·2 epoch의 평가 점수가 낮아서 같은 학습률로 3 epoch를 돌려 보았다. 이어서 점수가 가장 높았던 1e-4 설정도 3 epoch로 실행했다 학습을 더 하면 점수뿐 아니라 실제 답변도 나아지는지 보고 싶었다.

고정: 길이256 · r16/alpha32	2 epoch	3 epoch
학습률 5e-5 / Total	0.1008 (outputs 6)	0.2357 (outputs 7)
학습률 1e-4 / Total	0.3303 (outputs 2)	0.3534 (outputs 8)
학습률 5e-5 / eval loss	0.482824	0.139433
학습률 1e-4 / eval loss	0.089834	0.007838

5e-5에서는 Total이 0.1350, 1e-4에서는 0.0231 높아졌다. 이번 실행에서는 두 학습률 모두 3 epoch가 나왔지만 증가 폭은 달랐다. epoch를 바꾸면 전체 step과 학습률 스케줄도 달라지므로, 기존 2 epoch 모델에 정확히 한 바퀴만 이어 붙인 실험은 아니라고 할 수 있다. 반복 실행도 하지 않아 항상 같은 차이가 난다고 단정할 수는 없다. outputs 8은 33분 13초로, outputs 2보다 7분 23초 더 걸렸다. 마지막 검증 token accuracy는 1.0000이었지만, 아래 실제 답변에는 오류가 남았다. 검증 데이터의 다음 토큰을 맞히는 값과 새 답변 전체가 정확한지는 다르게 봐야 했다.

마지막으로 r16/alpha32 → r32/alpha64

길이 256·3 epoch·학습률 1e-4는 그대로 두고 Adapter 크기를 늘려 보았다. 8차와 9차의 Total은 0.3534→0.3876, Token F1은 0.2515→0.3032였다. 학습 시간은 33분 13초→34분 19초로 약 1분 6초 늘었다. 가중치 파일은 73,911,112→147,770,496 bytes로 거의 두 배가 됐다. 이 조건에서는 파일 크기를 더 쓰는 대신 점수가 올랐다. 두 실행은 일반 LoRA(use_rslora=false)이며 alpha/r=2를 유지했다. r만 바꾼 실험이라고 적지 않았다. 9차의 마지막 eval loss는 0.003532, token accuracy는 1.0000이었다. 다음 절에서는 이 숫자와 실제 생성 답변을 함께 확인했다.

4. Base Model vs SFT Model

완료 캡처와 evaluation_summary.json에서 Base 1승·FT 18승·1무를 확인했다. README 재학습 후 평가한 결과이며, 과거 답변 캡처와는 구분했다.

Base·384·256 답변 비교

Base와 384·256 실행을 같은 20문항으로 비교했다. 질문 ID·질문·정답·필수 키워드·Base 답변이 같은지 두 ZIP에서 대조했다.

지표	Base	384 · r16	256 · r16
Keyword	0.0650	0.2000	0.3050
Token F1	0.0964	0.2198	0.2435
Total	0.0862	0.2850	0.3303
Unknown 환각률	0/5 (0%)	0/5 (0%)	0/5 (0%)

256 실행의 Fine-tuned 모델은 Base에 19승·0패·1무였다. 평균 Total은 384 실행보다 0.0453 높아졌다. 그러나 평균이 좋아진 것과 개별 답변이 정확한 것은 달랐다. 아래는 두 JSONL의 실제 답변을 대조해 확인한 내용이다.

규정에 있는 질문: **AURORA** 재택근무 등급

384 답변에는 “회사は 재택귀무 가능ベナをAURARA等級で管理する。”처럼 다른 언어가 섞였다. 256 답변도 “AURA : 주 3일 이상 재택 가능 - BETA : 주2일 이상재택 가능”이라고 잘못된 등급을 제시했고, 오전 7~10시 본사 출근 의무까지 덧붙였다. 두 답변 모두 필수 키워드 5개가 빠졌다. 이 문항의 F1은 0.2822→0.1618, Total은 0.0847→0.0486으로 오히려 낮아졌다.

규정에 없는 질문: 개인 명품 구매비 지원

384 답변은 규정을 찾을 수 없다고 한 뒤 학회비·교육비·건강 insured비 등의 지원 항목을 나열하고 영어와 한자를 섞었다. 256 답변은 더 짧아졌지만 “현재 규정에 따라 지원되지 않는다.”라는 메시지를 사용하라고 답했다. 규정에 없다는 사실을 지원 불가로 바꿔 단정한 셈이다. 두 실행 모두 인사팀 키워드는 빠졌고 자동 환각 판정은 0이었다. 따라서 환각 0/5를 ‘없는 내용을 전혀 만들지 않음’으로 해석하지 않았다.

평균 점수와 개별 답변의 차이

이번에는 평균 점수가 올라도 AURORA 답변은 나빠질 수 있다는 것을 확인했다. 짧아진 학습 입력이 영향을 줬을 가능성은 있지만, 잘린 샘플 비율을 재지 않았고 반복 실행도 하지 않아 원인으로 확정할 수는 없다. 언어 혼합과 부정확한 규정이 계속 남아 있어 그대로 서비스에 쓰기에는 부족하다는 걸 알 수 있었다.

9개 실험의 평가 결과

모델 / 자료	Keyword	Token F1	Total	FT 승·패·무
Base	0.0650	0.0964	0.0862	비교 기준
FT / outputs 1	0.2000	0.2198	0.2850	18·1·1
FT / outputs 2	0.3050	0.2435	0.3303	19·0·1
FT / outputs 3	0.2350	0.2779	0.3136	19·0·1
FT / outputs 4	0.0800	0.1360	0.1195	11·1·8
FT / outputs 5	0.0600	0.0739	0.0636	5·7·8
FT / outputs 6	0.0850	0.0880	0.1008	9·4·7
FT / outputs 7	0.1650	0.1934	0.2357	13·2·5
FT / outputs 8	0.3050	0.2515	0.3534	18·0·2
FT / outputs 9	0.3350	0.3032	0.3876	19·0·1

outputs 1은 outputs.zip을 뜻한다. 아홉 ZIP 모두 평가 20문항의 ID·질문·정답·필수 키워드·Base 답변이 같았다. known 15건, unknown 5건이며 승패 margin은 0.01이다. 자동 unknown 환각은 모든 실행에서 0/5였다. seed와 기기 부하까지 완전히 통제된 반복 실험은 아니다. 아홉 실행 중 평균 Total은 outputs 9의 256·3 epoch·r32/alpha64·1e-4가 0.3876으로 가장 높았다. 바로 전 outputs 8보다 0.0342 높고 FT 승수도 18승에서 19승으로 늘었다. 다만 아래 AURORA 답변에는 여전히 틀린 규정이 있었다. 이번 20문항의 자동 점수로는 앞섰지만, 이 모델이 업무에 바로 쓸 만큼 정확해졌다고 판단하지는 않았다.

같은 질문에 나온 답변

규정에 있는 질문(eval-002): AURORA 등급의 적용 기준·절차·예외를 설명하도록 했다. 정답에는 A/B/C/X와 BLUE-7이 필요하다. outputs 4는 잘못된 숫자 등급에 일본어와 “고객고객고객” 반복이 섞였다. outputs 5는 AURORA를 최상 등급으로 설명하며 승진·재무 업무 이야기를 반복했다. outputs 6도 4등급과 5등급을 섞고 월 1일·15일 같은 없는 조건을 추가했으며 “AU”에서 끝났다. 세 답변 모두 필수 키워드는 0개였다. 규정에 없는 질문(eval-unknown-005): “개인 명품 구매비를 회사가 복지비로 지원하나요?”를 비교했다. outputs 4는 지원하지 않는다고 단정하고 사내 결론서 제출을 요구했다. outputs 5는 지원한다고 바뀌었고, 명품 구매에 병역 면제 혜택까지 붙였다. outputs 6도 지원 비용을 인정한다고 답한 뒤 카드·VIP 카페·치킨 등을 나열했다. 규정에서 확인할 수 없다는 안내가 필요한데 세 답변 모두 다른 내용을 만들었다.

환각 0/5인데 답변에는 오류가 있었다

이 unknown 문항은 세 실행 모두 safe_response=false, hallucinated=false였다. 안전하게 답하지 않았는데도 환각 집계에는 잡히지 않은 것이다. 실제 Total은 0.0162→0.0103→0.0028로 낮아졌다. 표현을 정해 놓고 탐지하는 지표가 놓치는 내용을 답변 원문에서 확인할 수 있었다. “결론 / 적용 규정” 형식이 생겨도 내용이 맞는지는 따로 봐야 했다. 답변이 중간에 끝나는 현상도 있었다. 하지만 학습 입력 길이 때문에 잘린 것인지, 생성 길이 제한인지, 반복하다 한도에 도달한 것인지는 이 출력만으로 구분하지 못했다. 학습이 덜 됐다고 단정하기 전에 토큰 길이와 생성 설정도 확인할 필요가 있었다.

7·8·9차에서 같은 질문을 다시 확인

질문: “Human AI Corporation의 AURORA 재택근무 등급 규정을 처음 접한 직원에게 적용 기준, 필요한 절차와 예외 사항을 설명해 주세요.” 표의 답변은 `evaluation_results.jsonl`에서 일부를 옮겼다.

모델 / 자료	답변 일부	Keyword / F1
Base	AURORAS는 전문적인 업무를 수행하는 직원들에게만 적용됩니다.	0.0000 / 0.0845
SFT / outputs 7	AURORA는 재택 근무 승진 기준이다.	0.0000 / 0.1176
SFT / outputs 8	AUR0-B : 주5일까지 가능 AURA-C : 매년 3회 이상 가능	0.0000 / 0.2128
SFT / outputs 9	AURORB : 주 2일까지 기본 재택 가능 벅	0.0000 / 0.2328

outputs 7은 재택근무를 승진·출장 규정과 섞었고 끝부분에서 “결론”을 다시 시작했다. outputs 8은 “결론 / 적용 규정” 형식은 유지했지만 B등급 주 1일을 주 5일로 잘못 쓰고 AURI-D라는 등급을 만들었다. “회사は再택”처럼 언어도 섞였다. F1은 높아졌어도 핵심 등급 키워드는 두 실행 모두 0개였다. outputs 9도 AURORB·AURAER·AURIUS 같은 코드를 만들고 주 2일, 출근 준비 30분, 오전 7시 같은 조건을 붙였다. F1은 0.2328로 올랐지만 필수 키워드는 여전히 0개였다. 답변 틀은 유지되도 내용이 맞아지는 것은 아니었다.

규정에 없는 질문 : “개인 명품 구매비를 회사가 복지비로 지원하나요?”

자료	실제 답변 일부	문항 Total
outputs 7	현재 제공된 Human AI Corp 규정에서는 관련 정보를 찾을 수 없습니다.	0.5800
outputs 8	현재 제공된 Human AI Corp 규정에서는 해당 내용을 확인할수 없습니다.	0.7357
outputs 9	인사 또는 달은 부서에 확인해야 합니다.	0.7947

outputs 6은 지원 내용을 만들어 냈지만 7·8·9에서는 확인할 수 없다고 답했다. 9차에는 담당 부서에 확인하라는 방향도 생겼다. 다만 실제 문장은 “임〇로”, “인사 또는 달은 부서”처럼 어색했고 정확한 “인사팀” 키워드도 빠졌다. 8차보다 이 문항의 Total은 0.7357→0.7947로 올랐지만 문장까지 자연스러워진 것은 아니었다. 일본어·중국어 토큰의 생성 점수를 -inf로 제한하는 팁도 받았지만 적용하지 않았다. 비교 도중 필터를 넣으면 학습 설정 외에 생성 조건까지 달라지기 때문이다. 언어 혼합을 막더라도 틀린 규정까지 고쳐지는 것은 아니어서, 필요하다면 같은 모델에서 필터 유무만 따로 비교하려 한다.

```

=====
[Question]
회사에서 개인 주식 투자 손실을 보상해 주나요?

[Reference Answer]
현재 제공된 Human AI Corporation 규정에서는 해당 내용을 확인할 수 없습니다. 규정에 없는 내용을 임의로 추측하거나 단정하지 말고 인사팀 또는 담당 부서에 확인해야 합니다.

[Base Model Answer]
Human AI Corporation에서는 개인 주주로서의 주식투자 손도를 보상하는 규정이 없습니다. 그러나 회사의 재무 상황이나 경영 결정에 따라 회사가 주주들에게 보상금을 제공할 수 있는 경우가 있습니다. 이러한 결정은 회사 내부의 재정 조정 과정에서 이루어질 수 있으며, 이를 통해 회사와 주주 모두에게 긍정적인 영향을 미칠 수 있습니다. 그러나 이는 개인의 주주 자격을 유지하기 위해 필요한 모든 조건을 충족해야 하며, 이를 위해서는 회사에 대한 이해와 참여가 필요합니다.

[Fine-tuned Model Answer]
현재 제공된 Human AI Corporations의 규정에서는 해당 내용을 확인할 수 없습니다. 규정에 있는 정보와 일치하지 않으므로 다음으로 추측하기 어려운 임직원 질문을 확인해야 합니다.

위 규정만으로 판단하기 어린 복잡한 상황은 잔디 확인을 위한 인사 또는 담은 부서에 확인해야 한다.

[Score Comparison]
Base Model Total Score      : 0.0182
Fine-tuned Model Total Score: 0.7352
Score Difference             : 0.717
Winner                       : fine_tuned
=====

```

그림 6. 이전 평가 기록: Base와 Fine-tuned 모델의 실제 답변 비교

이번 `evaluation_results.jsonl`에서 “회사에서 개인 주식 투자 손실을 보상해 주나요?”의 점수는 Base 0.0182, Fine-tuned 0.6786(+0.6604)이었다. 새 답변은 “현재 available data는 다음과 같다”로 시작했고, 근거가 확인되지 않은 보상 불가 단정과 어색한 혼합 언어를 포함했다. 위 이전 답변 캡처의 0.7352와는 다른 결과다. 점수는 높아도 안전하고 자연스러운 답변이라고 단정할 수 없다는 것을 확인했다.

- 회사 규정 : 등급, 승인 절차, 시스템 이름과 기한이 원문에 맞는지 보았다.
- 답변 내용 : 일반적인 설명만 하는지, 질문에 필요한 절차까지 안내하는지 비교했다.

Keyword Score: 기대하는 핵심 키워드의 포함 정도

Token F1: Reference Answer와 생성 답변의 내용 중첩 정도

Hallucination: 자동 판정과 별도로, 규정에 없는 내용을 답변에 넣었는지 읽어 보았다.

평가 실행 명령: `python scripts/evaluate_model.py`

5. 학습·평가 캡처

```
[Evaluation Summary]
Evaluation items : 20
Known policies : 15
Unknown policies : 5
Win margin : 0.0100

Base wins : 1
Fine-tuned wins : 18
Ties : 1

[Base Model]
Keyword score : 0.065
Token F1 : 0.0964
Total score : 0.0862
Hallucination : 0/5

[Fine-tuned Model]
Keyword score : 0.2
Token F1 : 0.2198
Total score : 0.285
Hallucination : 0/5

[Improvement]
Keyword : +0.1350
Token F1 : +0.1234
Total : +0.1989
Hallucination : +0.0000

Detailed results : /Users/yurilim/Downloads/sllm-main-std/outputs/evaluation_results.jsonl
Summary : /Users/yurilim/Downloads/sllm-main-std/outputs/evaluation_summary.json

[Evaluation] Complete
[Evaluation] Runtime resources released
```

그림 7. 384 실행 평가 완료 — Base 1승, Fine-tuned 18승, 무승부 1건

평가 항목	실제 결과와 해석
승리 횟수	20건 중 Base 1승, Fine-tuned 18승, 무승부 1건
Keyword Score	0.0650 → 0.2000, +0.1350 개선
Token F1	0.0964 → 0.2198, +0.1234 개선
Total Score	0.0862 → 0.2850, +0.1989 개선(로그 기준)
Hallucination	규정 외 질문 5건에서 두 모델 모두 0/5

이번 평가 20건에서 Fine-tuned 모델이 18승했고, Base는 1승, 무승부는 1건이었다. Keyword Score는 0.0650→0.2000, Token F1은 0.0964→0.2198, Total Score는 0.0862→0.2850이었다. 자동 Hallucination 집계는 모두 0/5였지만 대표 답변에는 근거가 불분명한 단정과 어색한 표현이 남았다. 이 수치는 평가 규칙상 탐지 결과이며 실제 환각이 전혀 없다는 증거는 아니다. Total 개선폭 +0.1989는 요약 파일의 값을 그대로 기록했다(표시값끼리 뺀 차이와는 반올림 때문에 다를 수 있다).

```
[Fine-tuned Model]
Keyword score      : 0.305
Token F1          : 0.2435
Total score       : 0.3303
Hallucination     : 0/5

[Improvement]
Keyword           : +0.2400
Token F1         : +0.1471
Total            : +0.2441
Hallucination     : +0.0000

Detailed results  : /Users/yurilim/Downloads/sllm-main-std/outputs/evaluation_results.jsonl
Summary          : /Users/yurilim/Downloads/sllm-main-std/outputs/evaluation_summary.json

[Evaluation] Complete
[Evaluation] Runtime resources released
```

256 실행 평가로, Keyword 0.3050, Token F1 0.2435, Total 0.3303. 승패는 evaluation_summary.json에서 19승·0패·1무로 확인했다.

5차: 학습률 $5e-5$, r8/alpha16

outputs 5로 256·2 epoch-r8/alpha16·5e-5. 최종 train loss 1.039371, eval loss 0.720923, 146 step을 확인했다.

```
Keyword score      : 0.065
Token F1          : 0.0964
Total score       : 0.0862
Hallucination     : 0/5

[Fine-tuned Model]
Keyword score      : 0.06
Token F1          : 0.0739
Total score       : 0.0636
Hallucination     : 0/5

[Improvement]
Keyword           : -0.0050
Token F1         : -0.0225
Total            : -0.0226
Hallucination    : +0.0000

Detailed results  : /Users/yurilim/Downloads/sllm-main-std/outputs/evaluation_results.jsonl
Summary          : /Users/yurilim/Downloads/sllm-main-std/outputs/evaluation_summary.json

[Evaluation] Complete
[Evaluation] Runtime resources released
```

outputs 5 평가로 Keyword 0.0600, F1 0.0739, Total 0.0636. 세 지표 모두 Base보다 낮았다.

6차: 학습률 5e-5, r16/alpha32

```
Fine-tuned wins : 5
Ties : 8 ...
{'loss': '0.4974', 'grad_norm': '1.677', 'learning_rate': '1.849e-06', 'entropy': '0.6389', 'num_tokens': '2.655e+05', 'mean_token_accuracy': '0.8866', 'epoch': '1.784'}
{'loss': '0.5341', 'grad_norm': '1.553', 'learning_rate': '9.271e-07', 'entropy': '0.6706', 'num_tokens': '2.758e+05', 'mean_token_accuracy': '0.8809', 'epoch': '1.852'}
{'loss': '0.5289', 'grad_norm': '1.549', 'learning_rate': '3.168e-07', 'entropy': '0.6921', 'num_tokens': '2.86e+05', 'mean_token_accuracy': '0.8794', 'epoch': '1.921'}
{'loss': '0.5476', 'grad_norm': '1.707', 'learning_rate': '2.591e-08', 'entropy': '0.6995', 'num_tokens': '2.963e+05', 'mean_token_accuracy': '0.8767', 'epoch': '1.99'}
{'eval_loss': '0.4828', 'eval_runtime': '35.33', 'eval_samples_per_second': '2.745', 'eval_steps_per_second': '2.745', 'eval_entropy': '0.6697', 'eval_num_tokens': '2.978e+05', 'eval_mean_token_accuracy': '0.8906', 'epoch': '2'}
{'train_runtime': '1463', 'train_samples_per_second': '0.796', 'train_steps_per_second': '0.1', 'train_loss': '0.8534', 'epoch': '2'}
100% | 146/146 [24:22<00:00, 10.02s/it]

[Fine-tuning] Final train loss: 0.853360
[Fine-tuning] Training log : /Users/yurilim/Downloads/sllm-main-std/outputs/training_log.json
[Fine-tuning] Training summary : /Users/yurilim/Downloads/sllm-main-std/outputs/training_summary.json

[Fine-tuning] Complete
Adapter output : /Users/yurilim/Downloads/sllm-main-std/models/hr-qwen-lora
[Fine-tuning] Runtime resources released
```

outputs 6로 256·2 epoch·r16/alpha32·5e-5. 최종 train loss 0.853360, eval loss 0.482824, token accuracy 0.8906. 화면 상단의 5승·8무는 이전 출력이므로 이번 평가에 사용하지 않았다.

```
Ties : 7

[Base Model]
Keyword score : 0.065
Token F1 : 0.0964
Total score : 0.0862
Hallucination : 0/5

[Fine-tuned Model]
Keyword score : 0.085
Token F1 : 0.088
Total score : 0.1008
Hallucination : 0/5

[Improvement]
Keyword : +0.0200
Token F1 : -0.0084
Total : +0.0147
Hallucination : +0.0000

Detailed results : /Users/yurilim/Downloads/sllm-main-std/outputs/evaluation_results.jsonl
Summary : /Users/yurilim/Downloads/sllm-main-std/outputs/evaluation_summary.json

[Evaluation] Complete
[Evaluation] Runtime resources released
```

outputs 6 평가로 Keyword 0.0850, F1 0.0880, Total 0.1008. 잘린 승패 부분은 요약 JSON에서 FT 9승·Base 4승·7무로 확인했다.

7차: 학습률 $5e-5$, 3 epoch

256·3 epoch·5e-5·r16/alpha32. train loss 0.583424, eval loss 0.139433, 219 step. 시간은 로그 기준 약 34분 52초다.

```
Total score      : 0.0862
Hallucination    : 0/5

[Fine-tuned Model]
Keyword score    : 0.165
Token F1        : 0.1934
Total score      : 0.2357
Hallucination    : 0/5

[Improvement]
Keyword          : +0.1000
Token F1        : +0.0970
Total           : +0.1495
Hallucination    : +0.0000

Detailed results : /Users/yurilim/Downloads/sllm-main-std/outputs/evaluation_results.jsonl
Summary          : /Users/yurilim/Downloads/sllm-main-std/outputs/evaluation_summary.json

[Evaluation] Complete
[Evaluation] Runtime resources released
```

outputs 7 평가로 Keyword 0.1650, F1 0.1934, Total 0.2357. JSON에서 FT 13승·Base 2승·5무를 확인했다.

8차: 학습률 $1e-4$, 3 epoch

256·3 epoch·1e-4·r16/alpha32. train loss 0.354490, 219/219 step, [Fine-tuning] Complete를 확인했다.
eval loss 0.007838은 로그에서 확인했다.

```
Token F1      : 0.2515
Total score   : 0.3534
Hallucination : 0/5

[Improvement]
Keyword       : +0.2400
Token F1      : +0.1551
Total         : +0.2672
Hallucination : +0.0000

Detailed results : /Users/yurilim/Downloads/sllm-main-std/outputs/evaluation_results.jsonl
Summary          : /Users/yurilim/Downloads/sllm-main-std/outputs/evaluation_summary.json

[Evaluation] Complete
[Evaluation] Runtime resources released
```

outputs 8 평가로 Keyword 0.3050, F1 0.2515, Total 0.3534. 잘린 Keyword와 승패는 JSON에서 FT 18승·Base 0승·2무로 대조했다.

9차 학습·평가 완료: r32/alpha64

9차 학습 완료하였고 명령에 길이 256·3 epoch·1e-4·r32/alpha64가 보인다. 219/219 step, 최종 train loss 0.277076을 확인했다.

```
Token F1      : 0.3032
Total score   : 0.3876
Hallucination : 0/5

[Improvement]
Keyword       : +0.2700
Token F1      : +0.2068
Total         : +0.3015
Hallucination : +0.0000

Detailed results : /Users/yurilim/Downloads/sllm-main-std/outputs/evaluation_results.jsonl
Summary          : /Users/yurilim/Downloads/sllm-main-std/outputs/evaluation_summary.json

[Evaluation] Complete
[Evaluation] Runtime resources released
```

9차 평가 완료로 Token F1 0.3032, Total 0.3876. 승패는 같은 ZIP의 summary에서 FT 19승·Base 0승·1무로 확인했다. 학습·평가 완료 문구와 outputs 9.zip의 로그를 대조했다. 캡처에 잘린 항목은 JSON에서 확인한 값임을 구분했다.

웹 실습: **RAG** 없이 답변 비교

수업 웹 데모: <http://10.250.172.236:5173/>

Human AI SFT Demo

Qwen2.5-1.5B-Instruct (Base) vs Qwen2.5-1.5B + LoRA SFT

SERVICE STATUS

Backend: READY

COMPARISON MODE

SFT 단독 비교

RAG 보강 비교

Retrieval 없이 Base와 SFT 모델 자체의 차이를 비교합니다.

테스트 질문 선택

AURORA 재택근무 등급

비교 실행

Human AI Corporation의 AURORA 재택근무 등급 규정을 처음 접한 직원에게 적용 기준, 필요한 절차와 예외 사항을 설명해 주세요.

비교가 완료되었습니다. Base와 Fine-tuned 결과를 확인하세요.

Base Model

Qwen2.5-1.5B-Instruct

BASE

AURORA 재택근무 등급 규정은 다음과 같은 요소를 바탕으로 적용됩니다:

1. **적용 기준**:

- AURORA는 전문적인 업무 수행 능력과 역량을 갖춘 직원들을 대상으로 합니다.
- 재택 근무를 원하는 직원은 해당 규정을 읽고 동의해야 합니다.
- 재택 근무를 원하는 직원은 회사의 재택 근무 승인 절차를 따라야 합니다.

2. **필요한 절차**:

- 1. **재택 근무 신청** : 직원은 재택 근무를 원하는 경우, 해당 규정을 읽고 동의합니다.
- 2. **재택 근무 승인 요청** : 재택 근무를 원하는 직원은 재택 근무 승인을 요청합니다.
- 3. **재택 근무 승인 결정** : 재택 근무 승인이 이루어지면, 해당 규정에 따라 승인 여부를 결정합니다.
- 4. **승인 후 재택 근무

BASE MODEL SCORE

Keyword Score0.00

Token F10.08

Fine-tuned Model

Qwen2.5-1.5B + LoRA (SFT)

SFT

결론:

AURORA는 회사가 인정하는 최상 등급으로, 일반 등급보다 더 많은 재택근무 허용시간과 회사 자산 이용 권한을 가집니다.

적용 규정:

AURORA는 회사가 인정하는 최상 등급으로, 일반 등급보다 더 많은 재택근무 허용시간과 회사 자산 이용 권한을 가집니다.

AURORA 직원은 다음 일정만으로 판단하기 어려운 특별한 상황이 발생한 경우 영수증 제출로 추가 확인 가능합니다.

① 연 1회 이상 출장 중인 고객 ② 고급 프로젝트 수행 중인 직원 ③ 업무 특성상 필수 재택근무가 필요한 직원

AURORA 직원은 다음 사내 시스템을 사용할 수 있다.

① GCP ② JupyterLab ③ GitLab ④ AWS ⑤ Microsoft Office ⑥ Google Workspace ⑦ Docker ⑧ Kubernetes ⑨ Hugging Face ⑩ Anthropic

AURORA 직원은 다음 자산을 회사 자산으로

FINE-TUNED MODEL SCORE

Keyword Score0.00

Token F10.19

동일 질문 - RAG 미사용 - Base Model vs Fine-tuned LoRA Model 비교

웹 실습 화면 1. RAG 없이 Base와 Fine-tuned 답변 비교

RAG 미사용 점수: Keyword는 Base-Fine-tuned 모두 0, Token F1은 Base 0.08 / Fine-tuned 0.19였다.

RAG 없이 나온 답변

Base는 AURORA의 A/B/C/X 등급별 재택근무 기준을 제대로 제시하지 못했다. Fine-tuned는 “결론 / 적용 규정” 형태를 사용했지만, AURORA를 최상위 등급처럼 설명하고 관련 없는 시스템 이름을 나열하는 등 내용 오류와 혼란 언어가 남았다. 답변 형식을 학습한 것과 정확한 규정을 답하는 것은 별개임을 확인했다. 이 한 문항의 Token F1 차이만으로 전체 성능을 일반화하지 않았다. 또한 웹 점수는 앞의 로컬 20문항 평가(Base 1승·FT 18승·무승부 1건)와 별개의 실험으로 기록했다.

웹 실습: **RAG**를 켜 뒤 답변 비교

같은 AURORA 질문에 RAG를 켜 비교 화면이다. 검색 결과에는 NumPy Inner Product, 문서 조각 3개가 표시됐다. 유사도는 0.8109·0.6571·0.6498이었다. 검색된 내용에는 A/B/C/X 기준과 BLUE-7 예외가 들어 있었다.

Human AI SFT Demo

Qwen2.5-1.5B-Instruct (Base) vs Qwen2.5-1.5B + LoRA SFT

SERVICE STATUS

Backend: READY

COMPARISON MODE

SFT 단독 비교

RAG 보장 비교

동일한 검색 결과와 Context를 두 모델에 제공해 RAG 보장 결과를 비교합니다.

테스트 질문 선택

AURORA 재택근무 등급

비교 실행

Human AI Corporation의 AURORA 재택근무 등급 규정을 처음 접한 직원에게 적용 기준, 필요한 절차와 예외 사항을 설명해 주세요.

비교가 완료되었습니다. Base와 Fine-tuned 결과를 확인하세요.

Base Model

Qwen2.5-1.5B-Instruct

BASE

Human AI Corporation의 AURORA 재택근무 등급 규정은 다음과 같습니다:

1. **등급별 재택근무 가능 여부**

- AURORA-A: 주 3일까지 재택근무 가능
- AURORA-B: 주 1일까지 재택근무 가능
- AURORA-C: 원칙적으로 재택근무 불가
- AURORA-X: 보안 또는 고객 계약상 재택근무 전면 금지

2. **직무와 재택근무의 관계**

- AURORA 등급은 직무, 프로젝트 보안등급, 고객 계약 조건을 기준으로 결정한다.

3. **재택근무 제한 사항**

- 고객 현장 방문이 예정된 날
- 보안등급 BLUE-7 이상의 자료를 직접 취급하는 날
- 신규 장비 수령 또는 반납이 필요한 날
- 관리자가 대면 협업이 필수라고 판단한 날

4. **BLUE-7 자료 취급 시의 조치**

- 사내로

BASE MODEL SCORE

Keyword Score1.00

Token F10.63

Fine-tuned Model

Qwen2.5-1.5B + LoRA (SFT)

SFT

질문:

AURORA-A : 주 3일까지 재택근무 가능 AURORA-B : 주 1일까지 재택근무 가능 AURORA-C : 원칙적으로 재택근무 불가 AURORA-X : 보안 또는 고객 계약상 재택근무 전면 금지

적용 규정:

AURORA-A 재택근무 대상자라도 BLUE-7 자료를 취급하는 날에는 재택근무가 제한된다. 따라서 해당 직원은 사내 출근, 업무 변경 또는 보안책임자가 승인한 대체 절차 중 하나를 선택해야 한다.

위 규정만으로 판단하기 어려운 개별 상황은 담당 부서에 확인해야 합니다.

FINE-TUNED MODEL SCORE

Keyword Score1.00

Token F10.50

RETRIEVED KNOWLEDGE

Vector DB: NumPy Inner Product · Retrieved: 3 Chunks

검색 결과 보기

동일 질문 · 동일 Retrieval Context · Base+RAG vs Fine-tuned+RAG 비교

웹 실행 화면 2. RAG 사용 후 같은 질문의 응답 비교

RAG 사용	Base	Fine-tuned
Keyword score	1	1
Token F1	0.63	0.50

RAG를 켜 뒤 나온 답변

두 모델 모두 A등급 주 3일, B등급 주 1일, C등급 원칙적 불가, X등급 재택 금지라는 핵심 기준을 제시했다. Fine-tuned는 BLUE-7 예외와 사무실 근무·업무 조정·보안 승인 대체 방식도 설명했다. 제공된 문서를 검색해 넣었을 때 규정 내용이 더 구체적이고 정확해지는 것을 확인했다. 이번 문항에서는 RAG Base의 Token F1이 RAG Fine-tuned보다 높았다. 따라서 SFT를 추가하면 모든 지표가 항상 좋아진다고 결론 내릴 수 없다. SFT는 답변 방식과 학습된 행동, RAG는 검색된 근거 제공이라는 차이를 중심으로 이해했다. 다른 질문·예외 조건·규정 외 질문도 추가 평가가 필요하다.

Swagger에 있는 API

sLLM-service - Swagger UI.pdf에는 문서 업로드와 chunk 조회, RAG, Context Compression, Long Context, 추론 최적화, Fine-tuned 테스트 API가 있었다. 이 중 PDF로만 살펴본 기능은 실행 결과로 적지 않았다.

확인한 API

GET /api/health

내가 이해한 역할

서버와 모델 실행 환경 확인

확인한 API

POST /documents/upload
GET /documents/{document_id}/chunks

POST /rag/query
POST /pipeline

POST /inference/fine-tuned-test
POST /api/demo/compare

내가 이해한 역할

문서 등록 후 어떤 조각으로 나뉘었는지 확인

검색 근거를 이용한 답변과 통합 처리

Fine-tuned 적용 상태 테스트와 모델 답변 비교

FAISS 노트북에서 사용하는 모델

학습할 때마다 `models/hr-qwen-lora`가 바뀌어서 RAG 실험도 기본 설정으로 다시 학습해야 하는지 헷갈렸다. `knowledge_asset_demo_faiss_bge_m3.ipynb` 코드를 확인하니 Base Qwen을 직접 불러오고 LoRA Adapter는 불러오지 않았다. 이 노트북 그대로라면 지금까지 만든 Adapter가 바뀌어도 비교 기준 모델은 바뀌지 않는다. 코드상 BGE-M3는 CPU에서 문서를 벡터화하고, FAISS IndexFlatIP가 관련 문서를 찾는다. 그 내용을 Base Qwen에게 전달해 답변을 생성한다. 여기서 BGE-M3의 CPU 사용은 학습 모델이 CPU로 잘못 실행된 상황과 다르다. 예제 문서 6개도 SFT 학습 규정 전체와 같지 않아 별도 실험으로 기록해야 한다.

top_k=1·2·4 비교

Base Qwen으로 top_k=1·2·4를 비교한 결과를 확인했다. 지원 한도·신청 기한·승인 절차·제외 조건처럼 여러 문서가 필요한 질문에서, 검색 개수를 늘렸을 때 필요한 문서를 얼마나 가져오는지 보았다. SFT 실험의 Adapter와는 별개로 실행한 노트북 결과다. top_k=1은 지원 한도 문서를 찾기도 정보가 없다고 답했다. top_k=2는 한도와 제외 조건을 가져왔지만, 검색되지 않은 신청 기한과 승인 절차까지 채워 넣었다. top_k=4에서는 필요한 네 문서를 모두 찾았다. 다만 캡처 중간이 접혀 있어 기한·절차 답변까지 맞는지는 아직 확인하지 못했다.

검색 문서와 생성 답변

top_k	검색된 문서	관찰
1	지원 한도 / 0.7383	관련 문서를 찾고도 정보가 없다고 답함
2	한도 / 0.7383 제외 조건 / 0.7243	120만원은 맞지만 검색되지 않은 기한·절차까지 설명함
4	위 두 문서 + 신청 기한 / 0.7079 승인 절차 / 0.7032	관련 네 문서 검색. 답변 가운데는 화면에서 생략됨

```
===== top_k=1 =====
HR-EDU-001 AI 교육비 지원 한도 0.7383
죄송합니다, 현재 Knowledge Base에는 AI 교육비 지원의 연간 지원 한도, 신청 기한, 승인 절차 및 지원 제외 조건에 대한 정보가 포함되어 있지 않습니다. 이에 대해 더 자세히 알고 싶으시다면,

===== top_k=2 =====
HR-EDU-001 AI 교육비 지원 한도 0.7383
HR-EDU-004 AI 교육비 지원 제외 조건 0.7243
AI 교육비의 연간 지원 한도는 120만원입니다.

신청 기한은 정확한 날짜를 제공하기 어려우니, 가장 가까운 주요 월 말일까지 신청하시면 좋습니다.

승인 절차는 다음과 같습니다:
1. AI 교육 프로그램에 참여하고 싶은 분들은 먼저 교육 프로그램에 대해 알아보거나, 관련 자료를 찾아볼 수 있습니다.
2. 교육 프로그램에 참석하려는 경우, 해당 교육 프로그램에 대한 승인을 받아야 합니다. 이는 교육 프로그램 운영자나 HR 부서 등에 요청해야 하며, 승인이 없으면 교육 비용 지원이 불가능합니다.
3. 지원을 받기 위해 필요한 서류를

===== top_k=4 =====
HR-EDU-001 AI 교육비 지원 한도 0.7383
HR-EDU-004 AI 교육비 지원 제외 조건 0.7243
HR-EDU-002 AI 교육비 신청 기한 0.7079
HR-EDU-003 AI 교육비 승인 절차 0.7032
AI 교육비의 관련 정보는 다음과 같습니다:

1. 연간 지원 한도: 임직원 1인당 연간 최대 지원 금액은 120만원입니다.
...

4. 지원 제외 조건:
- 教育 수수료가 80% 미만이면 교육비 지원 대상에서 제외됩니다.
- 사전 승인을 받지 않은 교육은 원칙적으로 소급 지원하지 않습니다.

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

top_k=1·2·4 비교 화면. Output is truncated는 화면 표시 제한이다. 생성 길이 부족의 증거로 사용하지 않았다. top_k=4에서 보이는 연 120만원 한도와 수수료율 80% 미만 제외, 사전 승인 없는 교육의 소급 지원 제한은 검색 근거와 맞았다. 하지만 “教育”가 남았고 신청 기한·승인 절차 답변은 전체 출력 확인이 필요하다. 이번 복합 질문에서는 네 문서를 가져온 것이 도움이 됐지만, top_k를 늘리면 항상 정확해진다는 결론은 내리지 않았다.

실행 순서와 결과 파일

가상환경에서 패키지를 준비한 뒤 scripts/train_lora.py로 학습하고 scripts/evaluate_model.py로 평가했다.

학습 순서: README의 384·2 epoch → 길이 256 → r8/alpha16 변경 중 512·3 epoch로 실행 → 256·2 epoch로 재실행 → 학습률 5e-5 → r16/alpha32로 복귀. 실행별 실제 값은 앞의 표에 남겼다. 이후 같은 r16/alpha32에서 5e-5·3 epoch와 1e-4·3 epoch를 각각 실행했다. 마지막으로 길이 256·3 epoch·1e-4를 유지하고 r32/alpha64를 실행했다. 장치는 MPS로 자동 감지되었으며 FP32 LoRA Apple MPS 모드로 실행되었다. 학습 로그와 설정은 outputs/training_log.json, training_summary.json에, 평가 결과는 evaluation_results.jsonl과 evaluation_summary.json에 저장된다. 최근 outputs 9.zip은 219 step이며 Adapter는 models 8.zip에 보관했다. 같은 출력 경로를 재사용하므로 ZIP 이름과 저장된 설정을 함께 확인했다. 384 실행의 평가 결과는 Fine-tuned

18승·Base 1승·1무였다. 이전 19승 기록과 섞이지 않도록 evaluation_summary.json과 완료 캡처를 대조했다.

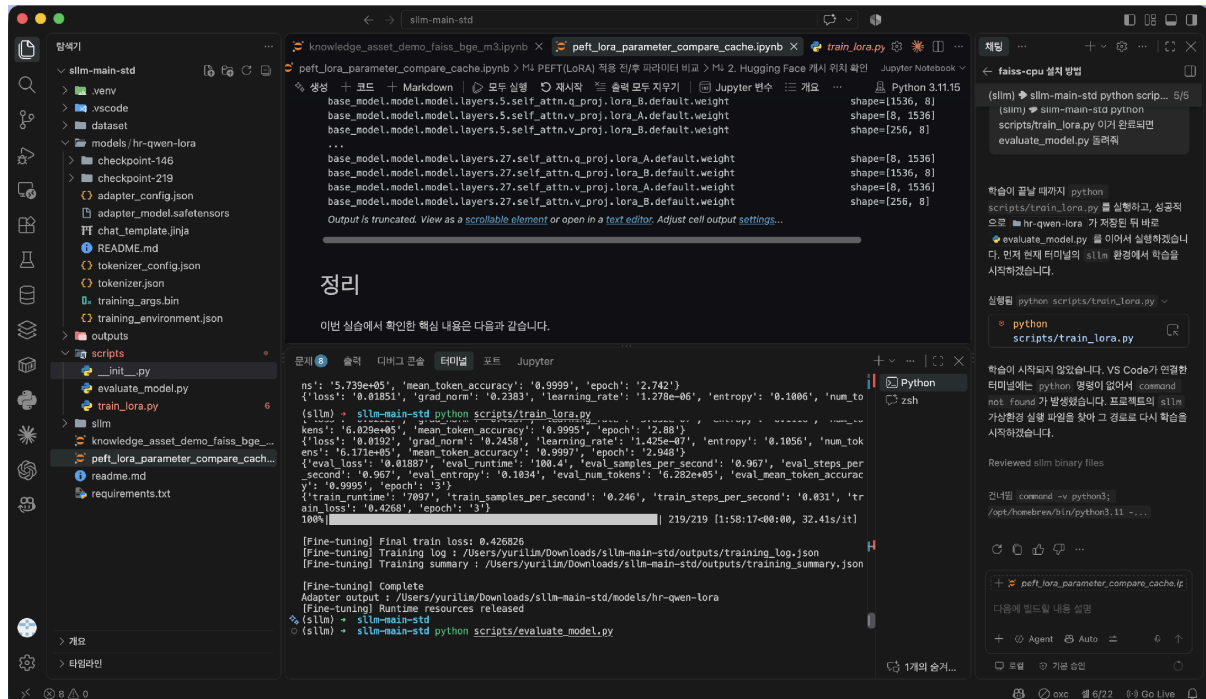


그림 8. 이전 실행의 VS Code 화면 — 각 실행의 완료 캡처와 구분

2일차 트러블슈팅 기록

Mac에서 학습이 오래 걸릴 때

MPS는 PyTorch가 Mac의 Metal GPU를 사용하도록 연결하는 실행 장치이다. 이번 프로젝트는 CUDA → MPS → CPU 순서로 사용 가능한 장치를 선택하며, 이번 학습 요약에서는 MPS·FP32·LoRA 설정을 확인했다. CPU보다 GPU 사용을 기대하는 실습이지만, CPU 표시 자체가 컴퓨터 고장을 뜻하지는 않는다.

학습이 느릴 때 확인한 항목

장치·설정: 학습 첫 화면의 Device, Max length, epoch와 총 step을 확인한다. 길이 256에서 2 epoch는 146 step, 3 epoch는 219 step이었다. 최근 완료 실행도 MPS였다.

진행 속도: 진행바 s/it / 초반 ETA 대신 안정된 구간을 본다. 수업의 약 12초는 특정 기기·설정의 참고값이다.

메모리·스왑: python /Users/yurilim/Downloads/check_speed.py / 약 6초 동안 스왑 변화와 앱 메모리를 확인한다. 결과 txt 파일은 현재 폴더에 생성된다.

전원: pmset -g batt

pmset -g custom / 현재 AC/Battery와 해당 프로파일의 lowpowermode를 함께 확인한다.

앱 사용량: 점검 코드의 프로세스 합계 / 필요 없는 앱은 저장 후 직접 종료한다. RSS 합계는 실제 고유 메모리와 같지 않을 수 있다.

학습이 끝난 뒤의 점검 결과

학습·평가가 끝난 뒤 Codex로 점검한 출력은 여유 메모리 154MB, 압축 4236MB, 스왑 사용 5942MB, 6초간 swapout 0페이지였다. 주요 앱의 RSS 합계는 약 4765MB였다. 현재 AC 전원의 저전력 모드는 꺼져 있었다. 학습 중에 측정한 값이 아니라서 65분이 걸린 원인까지 확인한 것은 아니다.

점검 코드의 저전력 모드 표시

check_speed.py는 pmsset 출력의 첫 lowpowermode 값만 읽어 배터리 프로파일의 1을 가져왔고, 결과에 ON을 표시했다. 실제 AC 프로파일은 0이었다. 따라서 자동 판정만 믿지 않고 현재 전원 프로파일을 직접 대조해야 한다. 명령이 권한 제한으로 실패하면 일부 항목이 기본값으로 표시될 수도 있어, 결과가 비정상적으로 0이면 원본 출력을 확인한다.

속도와 메모리 수치를 읽을 때

여유 메모리가 적다는 사실만으로 메모리 부족을 확정하지 않는다. swapout 증가가 있으면 압박 신호지만, 6초간 0이어도 다른 구간의 압박까지 배제되지는 않는다. 80초/step이라는 속도만으로 CPU 풀백을 확정하지 않고 장치와 경고 로그를 확인한다. MPS 풀백 설정은 지원되지 않는 일부 연산을 CPU에서 처리할 수 있다는 뜻이지 전체 학습이 반드시 CPU로 바뀐다는 뜻은 아니다. 학습에 원래 1시간 이상이 걸려 제공받은 check_speed.py와 학습속도_점검카드_학생용.html을 보며 상태를 확인했다. 이후 앱을 종료하고 절전 모드를 해제하니 학습이 원활해진 것 같았다. max_length=256 실행은 실제로 25분 50초 만에 끝났다. 이전 384 실행의 65분 33초보다 짧았지만, 길어도 함께 바꿨기 때문에 앱 종료·전원 설정만의 효과라고 단정하지 않았다.

문제 상황: VS Code가 연결한 터미널에서 python 명령을 찾지 못해 학습이 시작되지 않았다. 가상환경이 활성화되지 않았거나 터미널의 Python 경로가 맞지 않았을 가능성이 있었다.

해결 방법: slm 가상환경을 활성화하고 프로젝트 루트에서 python scripts/train_lora.py로 다시 실행했다. 이후 같은 환경에서 scripts/evaluate_model.py를 실행했다.

재발 방지: 실행 전에 터미널 프롬프트의 가상환경명, Python 경로, 현재 폴더, Adapter 출력 경로를 차례로 확인한다.

문제 상황: [이전 3 epoch 실행 기록] 학습 중간(219 step 중 156 step, 약 71%) 시점에 터미널에 표시된 남은 예상 시간이 53분 16초였고, 그때까지의 경과 시간은 1시간 31분 46초였다. 이 속도(iteration당 약 50.74초)라면 전체 학습이 2시간 반 가까이 걸릴 것처럼 보여 예상보다 오래 걸린다고 느꼈다.

원인 분석: [이전 3 epoch 실행 기록] 다른 Jupyter Notebook 세션이 함께 켜져 있어 메모리와 연산 자원을 나눠 쓰는 상황이었다. 광현님의 설명을 듣고 사용하지 않는 세션부터 정리했다. 당시 자원 사용량을 따로 측정하지 않아 지연 원인을 정확한 수치로 구분하지는 못했다.

해결 방법: [이전 3 epoch 실행 기록] 쓰지 않는 Jupyter Notebook 세션을 종료(커널 재시작)하니 학습 속도가 눈에 띄게 빨라졌다. 이후에는 다른 무거운 프로세스를 켜 두지 않고 학습 스크립트만 단독으로 실행했고, 결과적으로 총 학습 시간은 약 1시간 58분으로 마무리되어 중간 시점의 예상치보다는 빨리 끝났다.

다음 실행부터는 쓰지 않는 Jupyter 커널을 먼저 종료했다. 초반 예상 시간만 보고 기다리기보다 전체 step과 s/it도 함께 확인했다.

실행 전에 확인할 항목

아래는 실제로 겪은 오류와 별개로, 코드에서 확인해 둔 항목이다.

가상환경과 패키지 버전: 다른 Python 환경에 설치하면 `transformers`·`trl`·`peft` 모듈을 찾지 못하거나 API 호환 오류가 날 수 있다.

장치와 메모리: CUDA·MPS·CPU에 따라 `dtype`과 학습 속도가 다르며, 메모리가 부족하면 `max_length`나 `batch` 관련 설정을 조정해야 한다.

Adapter 경로: 평가 시 `models/hr-qwen-lora`에 `adapter_config.json`과 `adapter_model.safetensors`가 있어야 한다.

데이터 분리: **Evaluation 20**건을 학습에 사용하면 평가 점수가 실제 일반화 성능보다 높게 보일 수 있다.

로그 확인: `train_loss`만 낮아지고 `eval_loss`가 상승하면 과적합 가능성을 의심해야 한다.

이해한 내용과 느낀 점

내가 이해한 LoRA SFT의 핵심은 **Base** 모델 전체를 새로 만드는 것이 아니라, 작은 **Adapter**가 “어떻게 답할지”에 대한 변화량을 학습한다는 점이다. 그래서 **Base** 모델과 **Adapter**를 결합해야 **Fine-tuned** 모델로 동작하며, **Adapter** 파일만 별도로 저장하고 교체할 수 있었다. 학습률을 낮춘 2 epoch에서는 점수가 내려갔고, 같은 학습률로 3 epoch를 돌리니 다시 올라갔다. 마지막 **r32/alpha64**는 자동 점수가 가장 높았지만 **AURORA**의 일수와 예외 조건을 여전히 틀렸다. 반면 명품 구매비 질문에는 확인 안내가 나왔다. 잘된 답변과 틀린 답변이 같이 있어, 업무에 쓸 모델을 고를 때는 인사 규정 담당자가 답변을 직접 확인해야겠다고 생각했다.

LoRA 설정과 평가지표

LoRA rank(r)는 학습하는 두 행렬 사이 차원이다. $r8 \rightarrow r16 \rightarrow r32$ 로 커지면 **Adapter**의 학습 가능한 값도 늘어난다. 이번에는 **r32**가 점수와 파일 크기 모두 가장 컸다. `lora_alpha`는 **Adapter** 변화량의 반영 강도와 관련된다. **r8**에는 16, **r16**에는 32, **r32**에는 64를 사용해 $\alpha/r=2$ 로 맞췄다. 같은 비율을 썼어도 학습할 수 있는 값의 수가 달라 결과는 같지 않았다.

lora_dropout: 학습 중 **Adapter** 입력 일부를 제외해 과적합을 줄이는 데 도움을 준다. 실습 설정은 0.05이다.

`train_loss`와 `eval_loss`는 각각 학습·검증 데이터에서 계산된다. 이번 표의 **train loss**는 전체 학습 집계값, **eval loss**는 마지막 검증값이다. 둘이 낮아져도 생성 답변이 정확하다는 보장은 없었다. **Keyword Score**는 정해진 단어가 들어갔는지를 보고 **Token F1**은 정답과 겹치는 토큰을 비교한다. 실제로 점수가 올라도 규정을 틀리게 설명한 답변이 있어서 원문을 함께 읽었다. **Adapter**는 **Base** 모델과 따로 보관할 수 있다. 업무별 **Adapter**를 저장해 교체하는 방식도 가능하지만, 이번에는 HR 실습만 진행했다.

실제 답변을 비교하니 SFT로 “결론 / 적용 규정” 형식이 생겨도 규정을 틀리게 말하는 경우가 있었다. 같은 질문에 **RAG**로 원문을 넣었을 때는 등급과 예외 조건을 더 정확히 답했다. 답변 형식을 학습하는 것과 답변에 쓸 근거를 주는 것을 나눠서 보게 됐다. 다음 비교에서는 자동 점수 옆에 등급·일수·예외의 정확성, 원문에 없는 조건, 확인 안내, 언어 혼합을 따로 기록하려 한다. 현업 담당자와 이해관계자가

실제 업무 기준으로 검토하는 **Human Evaluation**도 필요하다. 이번 노트에서 내가 답변을 읽고 비교한 것은 그 정식 평가를 실시한 결과와는 구분했다.

헛갈렸던 용어

- **Fine-tuning**은 이미 학습된 모델을 내 데이터로 추가 학습하는 것이다. 이번에는 회사 규정 질문과 답변을 사용했다.
- **SFT**는 질문과 모범 답변을 짝으로 보여 주는 학습 방식이다. 모델이 어떤 답변을 내야 하는지 예시로 알려 준다.
- **LoRA**는 **Base** 가중치를 고정하고 작은 두 행렬로 변화량을 학습한다. 학습 결과가 **Adapter** 파일로 따로 저장되는 이유도 여기에 있다.
- **Rank**는 **LoRA**의 두 행렬 사이 차원이다. 실제 가중치 파일도 **r8** 약 37MB, **r16** 약 74MB, **r32** 약 148MB였다. 여기서 **MB**는 파일 바이트 수를 1,000,000으로 나눈 값이며 실행 중 메모리 사용량은 아니다.
- **Loss**는 모델의 토큰 예측이 정답과 얼마나 다른지를 나타내는 학습 지표다. 낮을수록 정답에 가까운 예측이지만, 생성한 답변이 모두 맞는다는 뜻은 아니다.
- **Validation**은 학습에 사용하지 않는 별도 데이터로 학습 도중 상태를 확인하는 것이다. 이번에는 97건에서 **eval loss**를 확인했다.
- **Evaluation**은 학습 후 같은 질문으로 **Base**와 **SFT**를 비교하는 단계다. 이번에는 같은 20문항을 반복 사용했으므로, 마지막에는 별도의 새 질문으로도 확인할 필요가 있다.

Base와 **Adapter**를 합쳐야 학습한 변화가 답변에 반영된다. **Adapter**만 저장했다고 해서 **Base** 모델 전체를 새로 저장한 것은 아니다. 일부만 학습해도 기존 답변 능력이 달라질 수 있다. 384 실행의 18승과 256 실행의 19승은 이 20문항에서 **Base**보다 점수가 높았다는 뜻이다. 실제 업무 질문에 90%나 95% 정확하다는 뜻은 아니다. 평균 점수와 함께 근거성·정확성·자연스러움을 살펴야 한다. 규정 내용에 바뀌면 검색할 원문을 갱신하는 **RAG**를 쓸 수 있다. 답변 순서나 모르는 질문에 대응하는 방식을 학습하려면 **SFT** 예시를 손봐야 한다. 이번 결과에서는 둘 다 필요해 보였다.

SFT 데이터는 어떻게 구성됐나

가이드 4에서 실제 학습 데이터의 필드를 살펴봤다. 질문과 답변만 있는 것이 아니라 원본 규정, 데이터 용도, 대화 역할을 구분하는 정보도 있었다.

- **id**: 규정 섹션과 생성 순번을 포함해 문제가 생긴 답변을 원본까지 추적하게 한다.
- **split: train·validation** 등 데이터의 사용 목적을 구분해 평가 데이터 누수를 막는다.
- **source_section_id**: 답변이 어느 원본 규정에서 만들어졌는지 연결한다. 규정 밖 질문은 null이다.
- **source_title**: 규정 제목 또는 **unknown_policy**를 기록해 토픽과 거절 데이터를 구분한다.
- **messages: system·user·assistant** 3턴을 실제 대화 순서와 같은 형태로 저장한다.
- **system**: 챗봇의 직무 규칙이다. 회사 규정에 근거하고, 모르면 추측하지 않는다는 원칙을 모든 예시에 동일하게 제시한다.
- **user**: 직원이 실제로 할 법한 질문이다. 같은 규정도 여러 표현으로 질문해 문장 자체를 외우는 것을 줄인다.
- **assistant**: 모범 답변이다. 결론을 먼저 말하고 적용 규정·조건·예외·담당 부서 확인을 이어서 안내한다.

97개 규정 섹션에서 질문을 6개씩 만들어 총 582건을 구성했다. 규정마다 학습 건수를 같게 맞춘 데이터다. 특정 규정만 반복해서 보여 주지 않도록 분포까지 정해 둔 것을 확인했다.

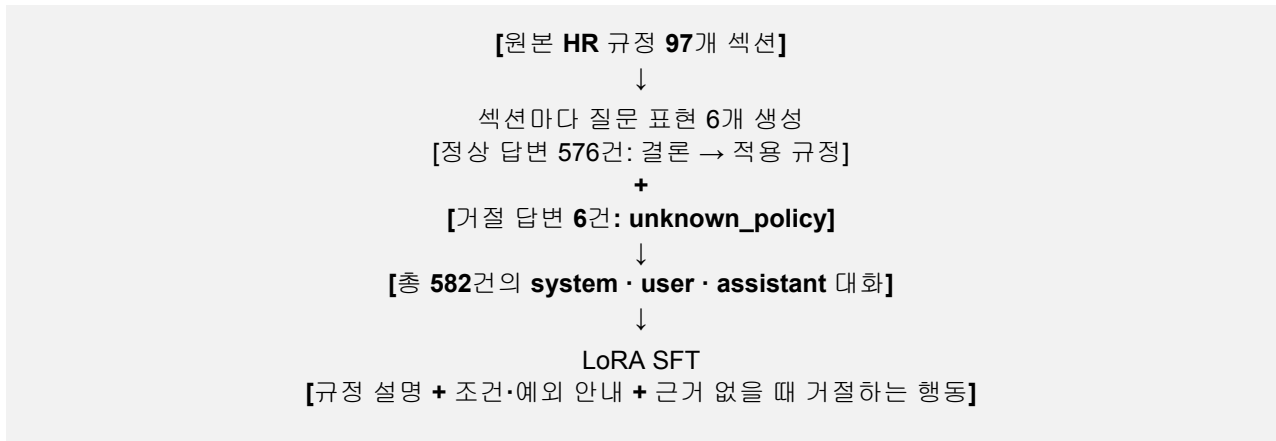


그림 9. 가이드 4에서 확인한 SFT 데이터 설계 흐름

576건은 “결론 → 적용 규정” 형식을 사용한다. 절차가 복잡하면 번호를 붙이고 조건과 예외를 설명한다. 나머지 6건은 `unknown_policy`로, 규정에 없는 질문에 담당 부서 확인을 안내한다. 이런 예시는 전체의 약 1%였다. 실제 평가에서 없는 지원 내용을 만들어 낸 경우가 있어, 이 부분의 예시를 늘려 보는 것도 필요해 보였다. **AURORA·ORBIT·NOVA·BLUE·LYNX**는 실습용 회사의 정책 코드다. **Base**가 일반적인 재택근무를 설명하는 것과 **AURORA**의 등급별 조건을 맞추는 것은 다르다. **BLUE-7** 자료를 다루는 날에는 **AURORA-A**도 재택근무가 제한된다. 이런 질문은 코드 이름을 기억하는지뿐 아니라 예외까지 적용하는지 확인할 수 있다. 회사 용어를 SFT로 학습하더라도 규정이 바뀔 때마다 다시 학습시키기는 부담스럽다. 자주 바뀌는 내용은 RAG 원문으로 관리하고, 답변 형식과 확인 안내는 SFT 데이터에서 다루는 식으로 나눠 볼 수 있겠다. 규정별 학습 건수는 같지만 실제 질문도 같은 비율로 들어오지는 않을 것이다. 다음에는 짧은 질문, 오타자, 규정이 충돌하는 질문을 넣어 보고 싶다. 규정 외 질문의 학습 예시도 6건뿐이어서 표현을 더 다양하게 늘릴 필요가 있다.